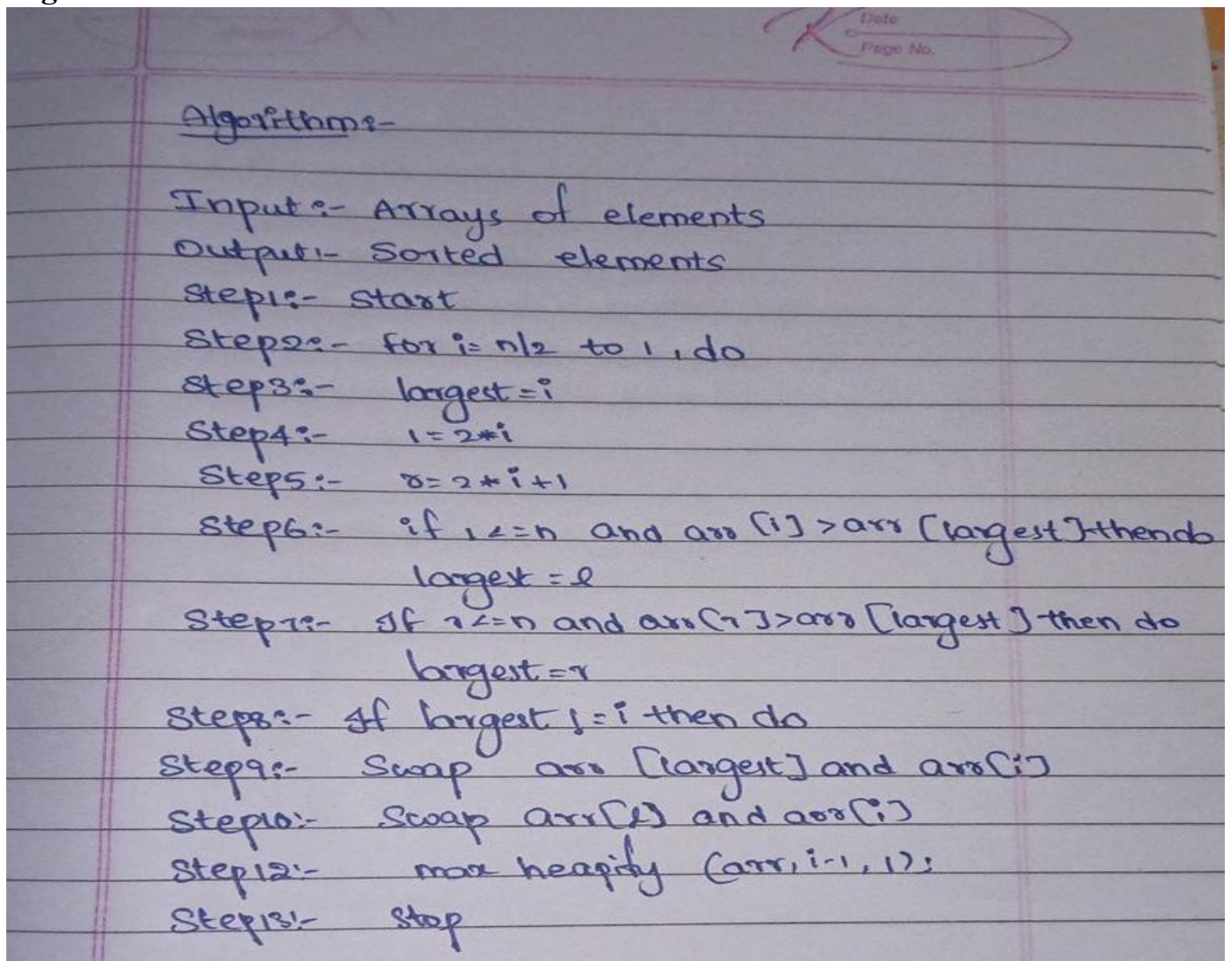


Practical: 3

Aim: To Implement of max-heap sort algorithms.

MAX-HEAP SORT :

Algorithm:



The image shows a handwritten algorithm for Max-Heap Sort on lined paper. The text is written in blue ink. At the top right, there is a circled area with 'Date' and 'Page No.' labels. The algorithm is titled 'Algorithm:-' and lists the following steps:

- Input:- Arrays of elements
- Output:- Sorted elements
- Step1:- start
- Step2:- for $i = n/2$ to 1, do
- Step3:- $largest = i$
- Step4:- $l = 2 * i$
- Step5:- $r = 2 * i + 1$
- Step6:- if $l \leq n$ and $arr[l] > arr[largest]$ then do
 $largest = l$
- Step7:- if $r \leq n$ and $arr[r] > arr[largest]$ then do
 $largest = r$
- Step8:- if $largest \neq i$ then do
- Step9:- Swap $arr[largest]$ and $arr[i]$
- Step10:- Swap $arr[l]$ and $arr[i]$
- Step12:- max heapify ($arr, i-1, l$)
- Step13:- stop

Program:

```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] >
arr[largest]) {        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i > 0;
i--) {        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
int main() {    int arr[] =
{4, 10, 3, 5, 1};    int n = 5;
heapSort(arr, n);    for (int i
= 0; i < n; i++) {        cout
<< arr[i] << " ";
    }
    return 0;
}
```

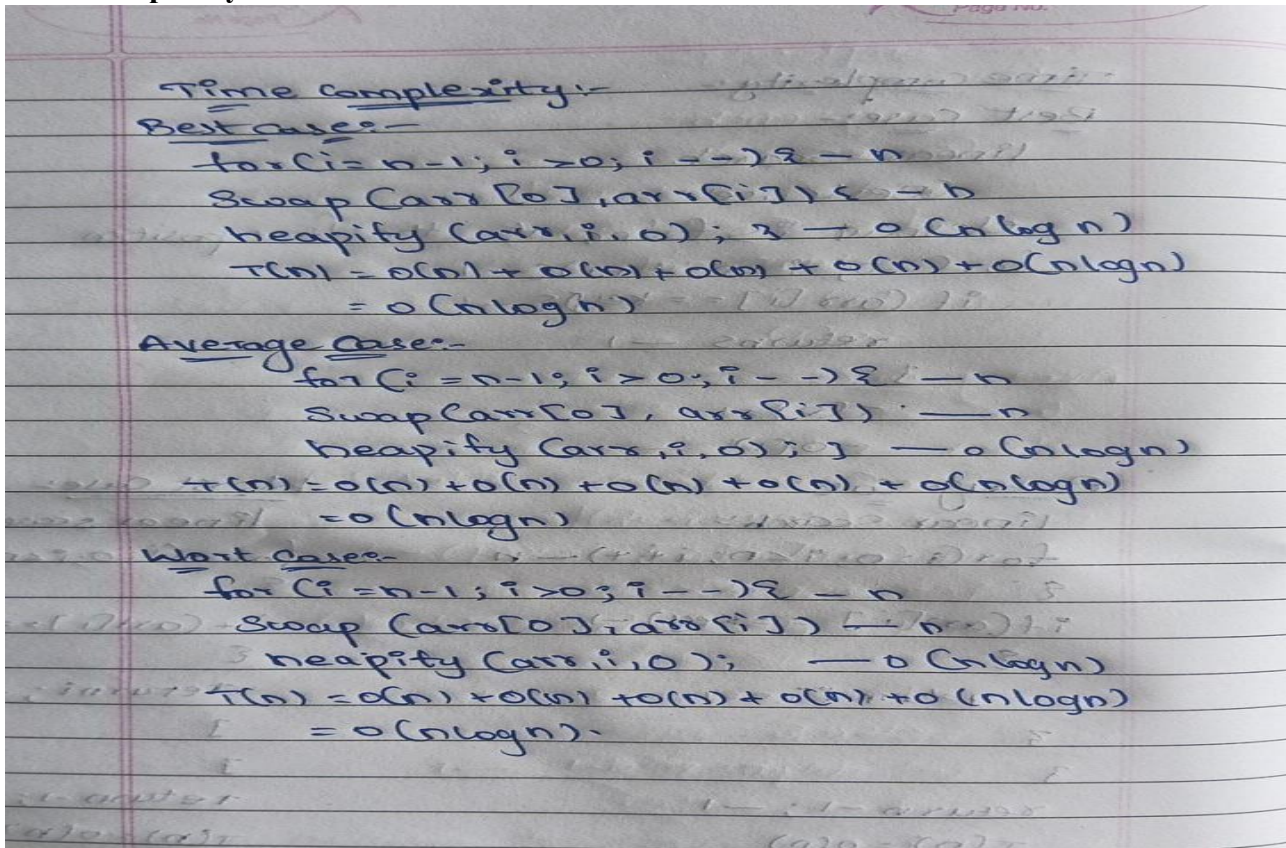
Output:

Output

```
Enter the number of elements: 5
Enter 5 elements:
3
1
6
4
2
Array after Heap Sort: 1 2 3 4 6

=== Code Execution Successful ===
```

Time Complexity:



Conclusion:

The implementation and time analysis of Max-Heap Sort helps us understand its sorting efficiency. Max-Heap Sort builds a Max Heap and repeatedly swaps the maximum element with the last element to sort the array in ascending order. It has a time complexity of $O(n \log n)$ in the best, average, and worst cases. Therefore, Max-Heap Sort is an efficient and consistent sorting algorithm for handling large datasets.

